



## help binsregselect

---

### Title

**binsregselect** — Data-driven IMSE-Optimal Partitioning/Binning Selection for Binscatter.

### Syntax

```
binsregselect depvar indvar [othercovs] [if] [in] [weight] [, deriv(v)
absorb(absvars) reghdfeopt(reghdfe_option)
bins(p s) binspos(position) binsmethod(method) nbinsrot(#)
nbins(nbinsopt)
pselect(numlist) sselect(numlist)
simsgrid(#) savegrid(filename) replace
dfcheck(n1 n2) masspoints(masspointsoption)
vce(vcetype) usegtools(on/off) precision(single/double) useeffn(#)
randcut(#) ]
```

where *depvar* is the dependent variable, *indvar* is the independent variable for binning, and *othercovs* are other covariates to be controlled for.

The degree of the piecewise polynomial *p*, the number of smoothness constraints *s*, and the derivative order *v* are integers satisfying  $0 \leq s, v \leq p$ , which can take different values in each case.

**fweights**, **aweight**s and **pweight**s are allowed; see [weight](#).

### Description

**binsregselect** implements data-driven procedures for selecting the number of bins for binscatter estimation. The selected number is optimal in minimizing the (asymptotic) integrated mean squared error (IMSE).

### Options

#### Estimand

---

**deriv**(*v*) specifies the derivative order of the regression function for estimation, testing and plotting. The default is **deriv**(0), which corresponds to the function itself.

#### Reghdfe

---

**absorb**(*absvars*) specifies categorical variables (or interactions) representing the fixed effects to be absorbed. This is equivalent to including an indicator/dummy variable for each category of each *absvar*. When **absorb**() is specified, the community-contributed command **reghdfe** instead of the command **regress** is used.

**reghdfeopt**(*reghdfe\_option*) options to be passed on to **reghdfe**. Important: **absorb**() and **vce**() should not be specified within this option.

For more information about the community-contributed command **reghdfe**, please see <http://scorreia.com/software/reghdfe/>.

#### Binning/Degree/Smoothness Selection

---

**bins**(*p s*) sets a piecewise polynomial of degree *p* with *s* smoothness constraints for data-driven (IMSE-optimal) selection of the partitioning/binning scheme. The default is **bins**(0 0), which corresponds to piecewise constant (canonical binscatter).

**binspos**(*position*) specifies the position of binning knots. The default is **binspos(qs)**, which corresponds to quantile-spaced binning (canonical **binscatter**). Other option is **es** for evenly-spaced binning.

**binsmethod**(*method*) specifies the method for data-driven selection of the number of bins. The default is **binsmethod(dpi)**, which corresponds to the IMSE-optimal direct plug-in rule. The other option is: **rot** for rule of thumb implementation.

**nbinsrot**(#) specifies an initial number of bins value used to construct the DPI number of bins selector. If not specified, the data-driven ROT selector is used instead.

**nbins**(*nbinsopt*) sets the number of bins for degree/smoothness selection. If **nbins(T)** is specified, the command selects the number of bins instead, given the specified degree and smoothness. If a numlist with more than one number is specified, the command selects the number of bins within this list.

**pselect**(*numlist*) specifies a list of numbers within which the degree of polynomial *p* for point estimation is selected.

**sselect**(*numlist*) specifies a list of numbers within which the number of smoothness constraints *s* for point estimation is selected. If not specified, for each value *p* supplied in the option **pselect()**, only the piecewise polynomial with the maximum smoothness is considered, i.e., *s*=*p*.

Note: To implement the degree or smoothness selection, in addition to **pselect()** or **sselect()**, **nbins(#)** must be specified.

---

#### Evaluation Points Grid Generation

---

**simsgrid**(#) specifies the number of evaluation points of an evenly-spaced grid within each bin used for evaluation of the supremum (infimum or *Lp* metric) operation needed to construct confidence bands and hypothesis testing procedures. The default is **simsgrid(20)**, which corresponds to 20 evenly-spaced evaluation points within each bin for approximating the supremum (or infimum) operator.

**savegrid**(*filename*) specifies a filename for storing the simulation grid of evaluation points. It contains the following variables: *indvar*, which is a sequence of evaluation points used in approximation; all control variables in *othercovs*, which take values of zero for prediction purpose; *binsreg\_isknnot*, indicating whether the evaluation point is an inner knot; and *binsreg\_bin*, indicating which bin the evaluation point belongs to.

**replace** overwrites the existing file when saving the grid.

---

#### Mass Points and Degrees of Freedom

---

**dfcheck**(*n1 n2*) sets cutoff values for minimum effective sample size checks, which take into account the number of unique values of *indvar* (i.e., adjusting for the number of mass points), number of clusters, and degrees of freedom of the different statistical models considered. The default is **dfcheck(20 30)**. See Cattaneo, Crump, Farrell and Feng (2025) for more details.

**masspoints**(*masspointsoption*) specifies how mass points in *indvar* are handled. By default, all mass point and degrees of freedom checks are implemented.

Available options:

**masspoints**(*noadjust*) omits mass point checks and the corresponding effective sample size adjustments.

**masspoints**(*nocheck*) omits within-bin mass point and degrees of freedom checks.

**masspoints**(*off*) sets **masspoints**(*noadjust*) and **masspoints**(*nocheck*) simultaneously.

**masspoints**(*veryfew*) forces the command to proceed as if *indvar* has only a few number of mass points (i.e., distinct values). In other words, forces the command to proceed as if the mass point and degrees of freedom checks were failed.

---

### Other Options

**vce**(*vcetype*) specifies the *vcetype* for variance estimation used by the command **regress** (or **reghdfe** if **absorb()** is specified). The default is **vce(robust)**.

**usegtools**(*on/off*) forces the use of several commands in the community-distributed Stata package **gtools** to speed the computation up, if *on* is specified. Default is **usegtools(off)**.

For more information about the package **gtools**, please see <https://gtools.readthedocs.io/en/latest/index.html>.

**precision**(*single/double*) controls the storage type used for internally generated Stata variables. The default is **precision(double)**, which uses **double** storage for temporary and generated variables to help align Stata results with the R and Python implementations. Specify **precision(single)** to preserve the legacy Stata implementation using **float** storage for those variables. Stata matrix and Mata computations remain double precision in either case.

**useeffn**(*#*) specifies the effective sample size *#* to be used when computing the (IMSE-optimal) number of bins. This option is useful for extrapolating the optimal number of bins to larger (or smaller) datasets than the one used to compute it.

**randcut**(*#*) specifies the upper bound on a uniformly distributed variable used to draw a subsample for bins selection. Observations for which **runiform()** ≤ *#* are used. *#* must be between 0 and 1.

### Examples

```
Setup
. sysuse auto
```

```
Select IMSE-optimal number of bins using DPI-procedure
. binsregselect mpg weight foreign
```

### Stored results

#### Scalars

|                          |                                                                     |
|--------------------------|---------------------------------------------------------------------|
| <b>e(N)</b>              | number of observations                                              |
| <b>e(Ndist)</b>          | number of distinct values                                           |
| <b>e(Nclust)</b>         | number of clusters                                                  |
| <b>e(deriv)</b>          | order of derivative                                                 |
| <b>e(imse_bsq_rot)</b>   | bias constant in IMSE, ROT selection                                |
| <b>e(imse_var_rot)</b>   | variance constant in IMSE, ROT selection                            |
| <b>e(imse_bsq_dpi)</b>   | bias constant in IMSE, DPI selection                                |
| <b>e(imse_var_dpi)</b>   | variance constant in IMSE, DPI selection                            |
| <b>e(nbinsrot_poly)</b>  | ROT number of bins, unregularized                                   |
| <b>e(nbinsrot_regul)</b> | ROT number of bins, regularized or user-specified                   |
| <b>e(nbinsrot_uknot)</b> | ROT number of bins, unique knots                                    |
| <b>e(nbinsdpi)</b>       | DPI number of bins                                                  |
| <b>e(nbinsdpi_uknot)</b> | DPI number of bins, unique knots                                    |
| <b>e(prot_poly)</b>      | ROT degree of polynomial, unregularized                             |
| <b>e(prot_regul)</b>     | ROT degree of polynomial, regularized or user-specified             |
| <b>e(prot_uknot)</b>     | ROT degree of polynomial, unique knots                              |
| <b>e(pdpi)</b>           | DPI degree of polynomial                                            |
| <b>e(pdpi_uknot)</b>     | DPI degree of polynomial, unique knots                              |
| <b>e(srot_poly)</b>      | ROT number of smoothness constraints, unregularized                 |
| <b>e(srot_regul)</b>     | ROT number of smoothness constraints, regularized or user-specified |
| <b>e(srot_uknot)</b>     | ROT number of smoothness constraints, unique knots                  |
| <b>e(sdpi)</b>           | DPI number of smoothness constraints                                |
| <b>e(sdpi_uknot)</b>     | DPI number of smoothness constraints, unique knots                  |

#### Matrices

|                            |                                                                           |
|----------------------------|---------------------------------------------------------------------------|
| <b>e(knot)</b>             | numlist of knots                                                          |
| <b>e(m_p)</b>              | vector of degrees of polynomial                                           |
| <b>e(m_s)</b>              | vector of number of smoothness constraints                                |
| <b>e(m_nbinsrot_poly)</b>  | ROT number of bins, unregularized, for each pair of degree and smoothness |
| <b>e(m_nbinsrot_regul)</b> | ROT number of bins, regularized or user-specified, for                    |

each pair of degree and smoothness  
**e(m\_nbinsrot\_uknot)** ROT number of bins, unique knots, for each pair of degree and smoothness  
**e(m\_nbinsdpi)** DPI number of bins, for each pair of degree and smoothness  
**e(m\_nbinsdpi\_uknot)** DPI number of bins, unique knots, for each pair of degree and smoothness  
**e(m\_imse\_bsq\_rot)** bias constant in IMSE, ROT selection, for each pair of degree and smoothness  
**e(m\_imse\_var\_rot)** variance constant in IMSE, ROT selection, for each pair of degree and smoothness  
**e(m\_imse\_bsq\_dpi)** bias constant in IMSE, DPI selection, for each pair of degree and smoothness  
**e(m\_imse\_var\_dpi)** variance constant in IMSE, DPI selection, for each pair of degree and smoothness

## **References**

Cattaneo, M. D., R. K. Crump, M. H. Farrell, and Y. Feng. 2024. On Binscatter. American Economic Review 114(5): 1488-1514.

Supplemental Appendix for On Binscatter. Supplemental Appendix.

Cattaneo, M. D., R. K. Crump, M. H. Farrell, and Y. Feng. 2026. Nonlinear Binscatter Methods. Review of Economics and Statistics, revise and resubmit.

Supplemental Appendix for Nonlinear Binscatter Methods. Supplemental Appendix.

Cattaneo, M. D., R. K. Crump, M. H. Farrell, and Y. Feng. 2025. Binscatter Regressions. Stata Journal 25(1): 3-50.

## **Authors**

Matias D. Cattaneo (maintainer). matias.d.cattaneo@gmail.com.

Richard K. Crump. richard.crump@gmail.com.

Max H. Farrell. mhfarrell@gmail.com.

Yingjie Feng. fengyingjiepk@gmail.com.